ASSESSMENT 10 | SOFTWARE ENGINEERING LAB

AI-Assisted Complaint Management for HMS

A practical machine-learning feature for complaint categorization and priority suggestions

BCSE301P • Hostel Management System

Priyasha Das • Shubham Mishra • Anushka Rani

Why add AI to the existing HMS?

Grounded in the project scope from Assignment 1

Current workflow

Students submit complaints; wardens track status as Pending, In Progress or Resolved.

Operational friction

Complaints can be vague, inconsistent, or difficult to triage quickly.

Proposed support

Suggest a complaint category and priority at submission; the warden remains in control.

- Fits the existing Complaint Management module—no need to redesign the whole system.
- Complements role-based access and complaint status tracking already specified.
- Does not change the existing Flask + MySQL architecture.

AI feature and expected behavior

The feature assists triage; it does not make final administrative decisions

1 Classify

Predict one category: Plumbing, Electrical, Mess/Food, Cleanliness, Noise/Discipline, Maintenance or Safety.

2 Suggest priority

Normal, Medium or High based on the predicted category and explicit urgency terms.

3 Explain

Return confidence and a short reason; student/warden can correct the suggestion.

Example input

“Power socket is sparking and I feel unsafe.”

Expected output

Electrical • High priority

The exact model output can vary; the urgency rule forces High when a configured urgent term is present.

How the AI works

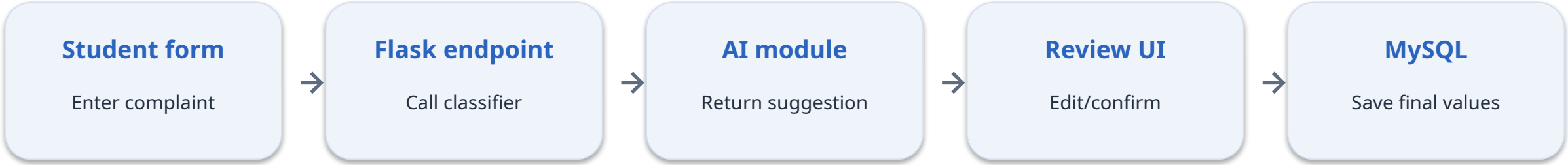
A small supervised text-classification prototype



- Training examples in this demo are synthetic and intentionally small.
- The classifier is trained locally at application startup; no external AI API is required.
- Confidence is a model score, not a guarantee of correctness.

Integration with the existing project

Designed around the current Flask + MySQL HMS stack



- Suggested API: POST /api/complaints/ai-suggestion with a description field.
- Response: category, priority, confidence, explanation.
- Existing authentication and role checks must still protect complaint routes.
- Persist only the user-confirmed result using the current complaint database logic.

Demonstration plan

Run the supplied prototype locally and show four representative inputs

Run

1. Install dependencies from requirements.txt
2. Run: python demo.py
3. Show each input and its predicted category/priority.

Example test cases

- Leaking pipe → Plumbing
- Sparking socket → High priority
- Spoiled food → Mess/Food
- Threat near gate → Safety / High priority

Live demo evidence to capture

- Terminal output from demo.py
- One complaint entered in the UI/API and the suggestion shown
- A corrected category/priority to demonstrate human control

This deck describes the supplied prototype. It does not claim that the AI has already been merged into the team's existing application.

Testing and evaluation

A credible demonstration needs both functional checks and model evaluation

Functional tests

Empty/short input rejected
Valid text returns expected fields
Urgency keywords yield High
Endpoint returns clear errors

Quality metrics

Evaluate on held-out, labelled complaints
Measure precision, recall and F1 per category
Review confusion matrix and false negatives

Human review

Allow correction
Record minimal audit data
Escalate safety cases
Never auto-close complaints

Important: the included synthetic training data is only for a working classroom demo. Do not present its outputs as measured real-world accuracy.

Privacy, security and limitations

Responsible integration matters more than adding an AI label

- Use only the existing authenticated complaint workflow; enforce server-side authorization.
- Do not send student names, room numbers or other identifiers to an external AI service.
- Treat complaint text as sensitive; define access, retention and deletion rules.
- Synthetic dataset is small and may not represent real student language or all complaint types.
- Model predictions can be wrong or biased; confidence is not certainty.
- High-risk safety complaints need direct human escalation, not an automated-only workflow.

No external service or student data is required for the local prototype.

Implementation roadmap

A realistic path from classroom prototype to a useful HMS feature

1	**Now**	Local prototype + CLI demo
2	**Integrate**	Connect the existing complaint form and Flask route
3	**Validate**	Test on anonymized, labelled complaint examples
4	**Deploy safely**	Human review, monitoring, access controls and rollback

Do not claim production readiness until real-data evaluation and integration tests are complete.

TAKEAWAY

AI should assist the warden, not replace the warden.

The proposed HMS Complaint Assistant demonstrates a practical ML integration: text classification, priority suggestions, explainable output, and human confirmation.

Thank you